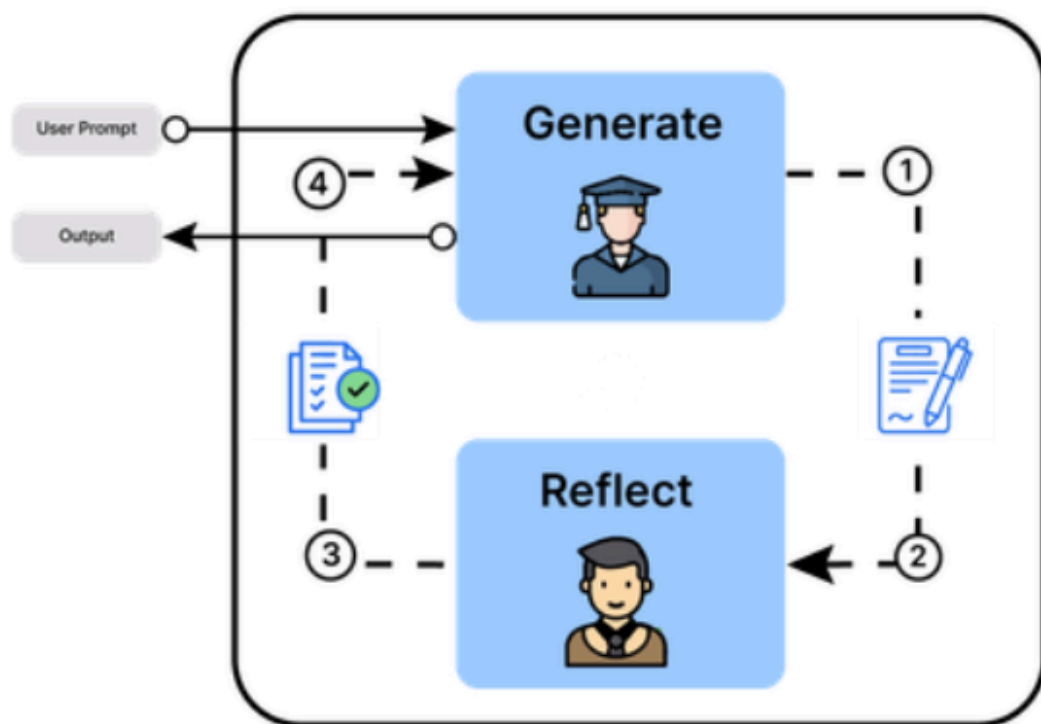
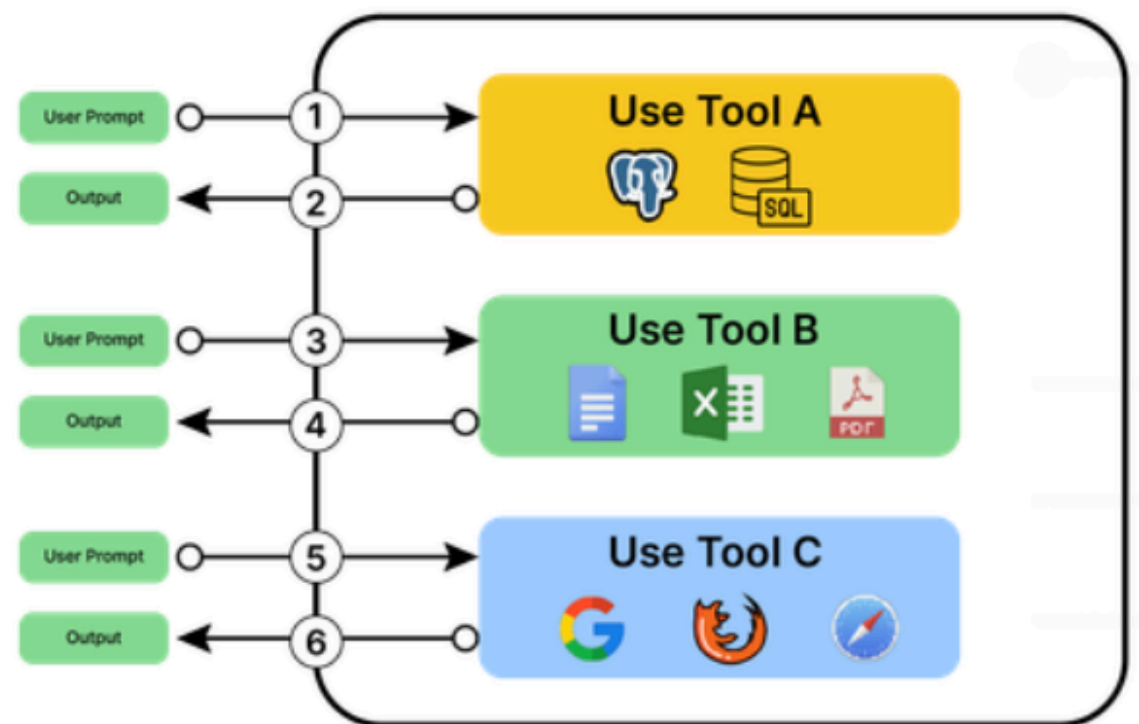


A Complete Guide to Agentic AI Design Patterns

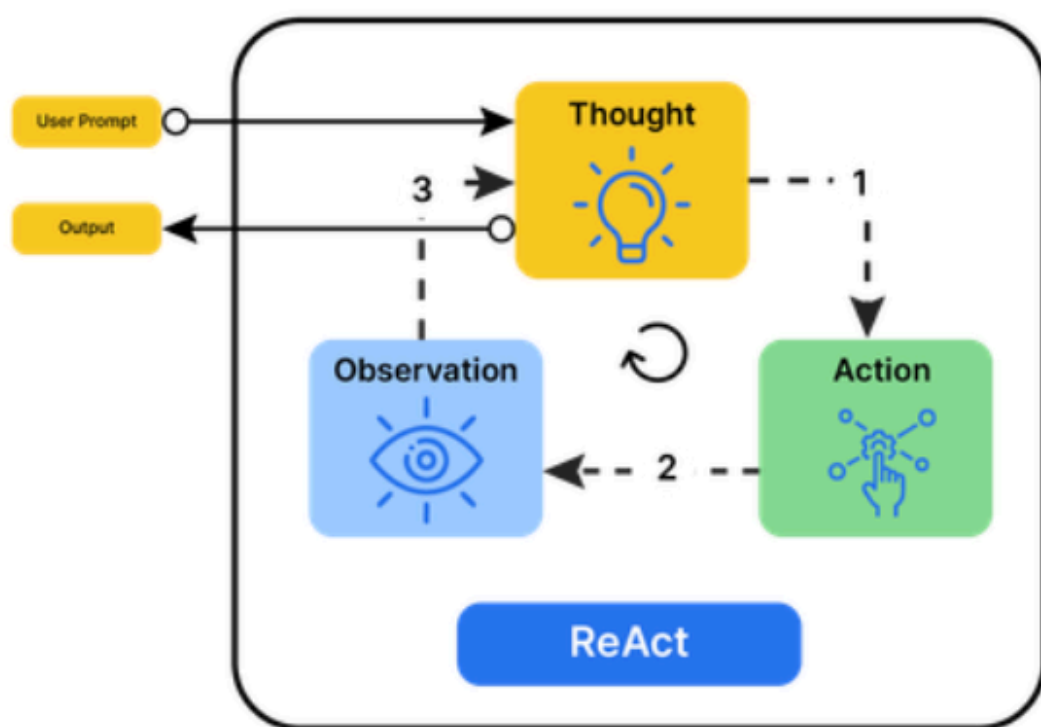
Reflection Pattern



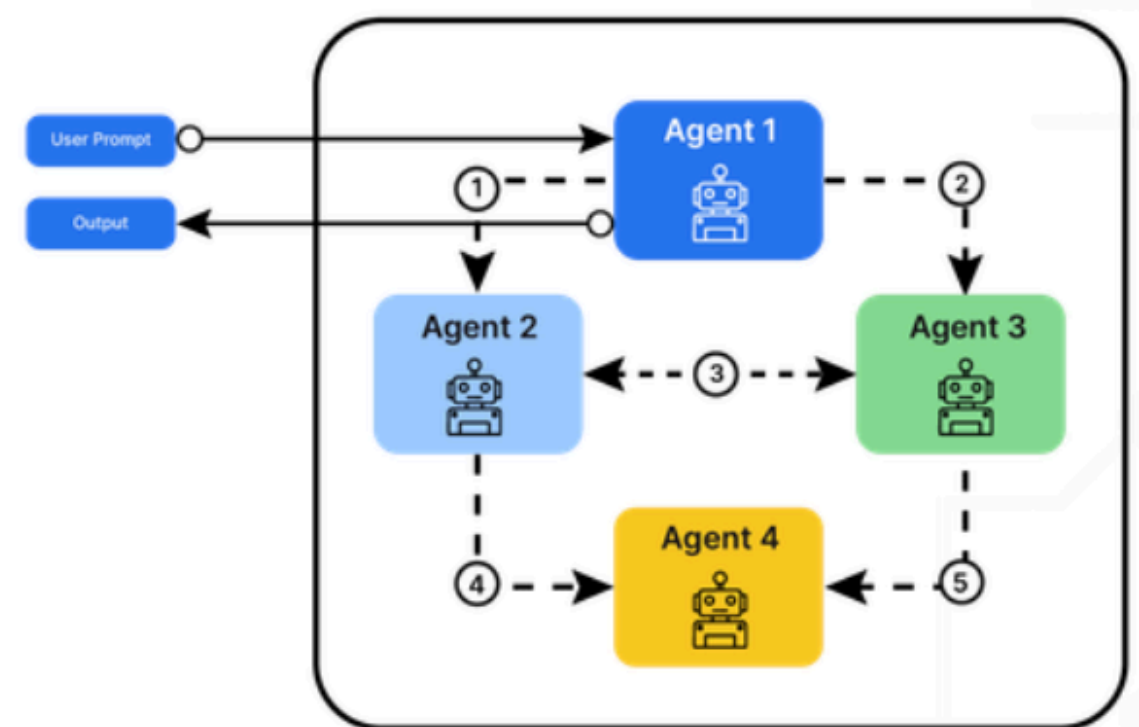
Tool Use Pattern



Planning Pattern



MultiAgent Pattern



What is Agentic Design Patterns?

The agentic design pattern is introduced as a solution for making LLMs more autonomous. Instead of just giving the model one prompt and expecting a final answer (like writing an essay in one go), an agent-like approach involves prompting the LLM multiple times, step by step.

Each step refines the task, with the model improving its output iteratively.

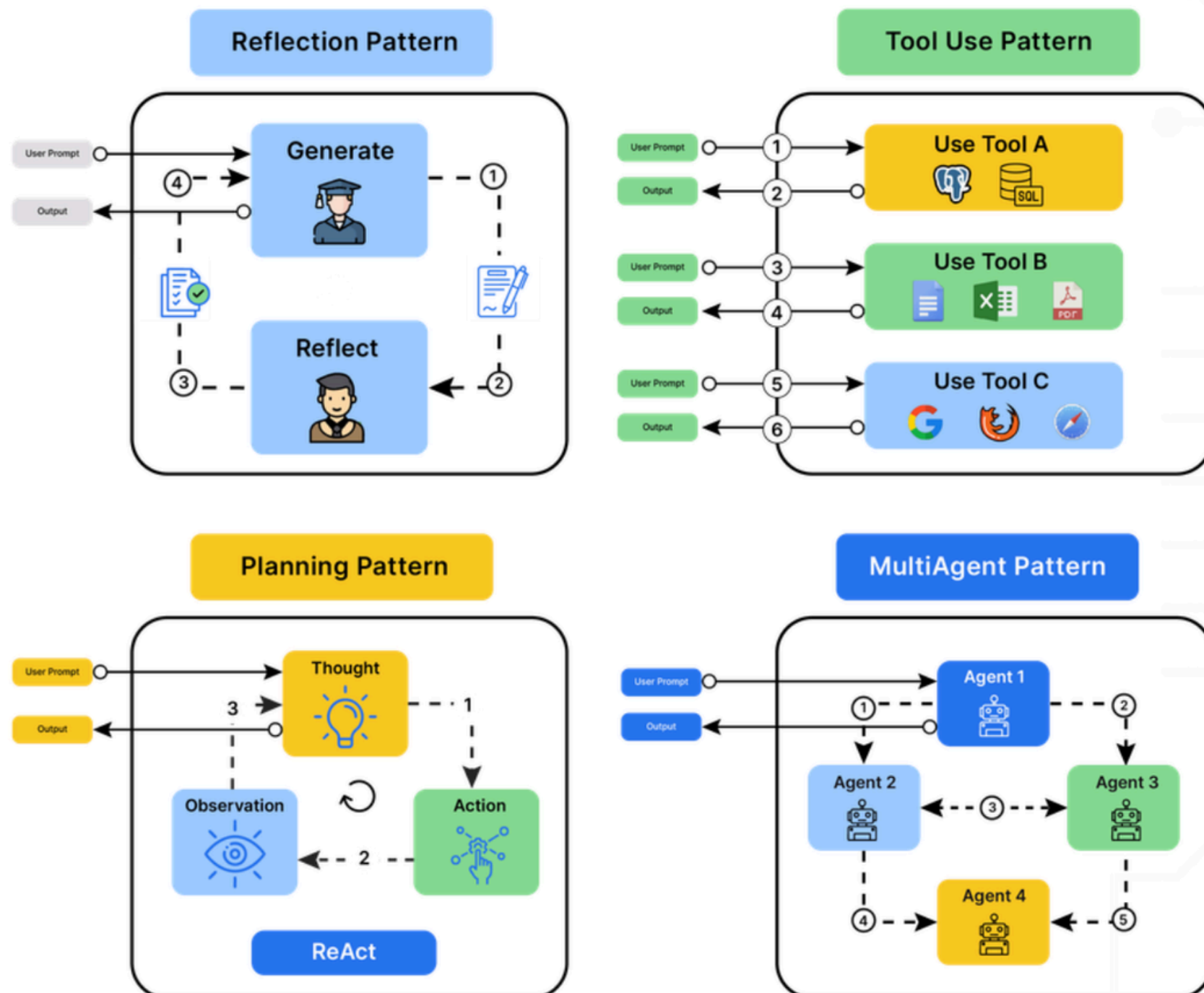
To understand this better, let's look at it like this:

When we prompt an LLM in zero-shot mode, it's like asking someone to write a story in one go without revising.

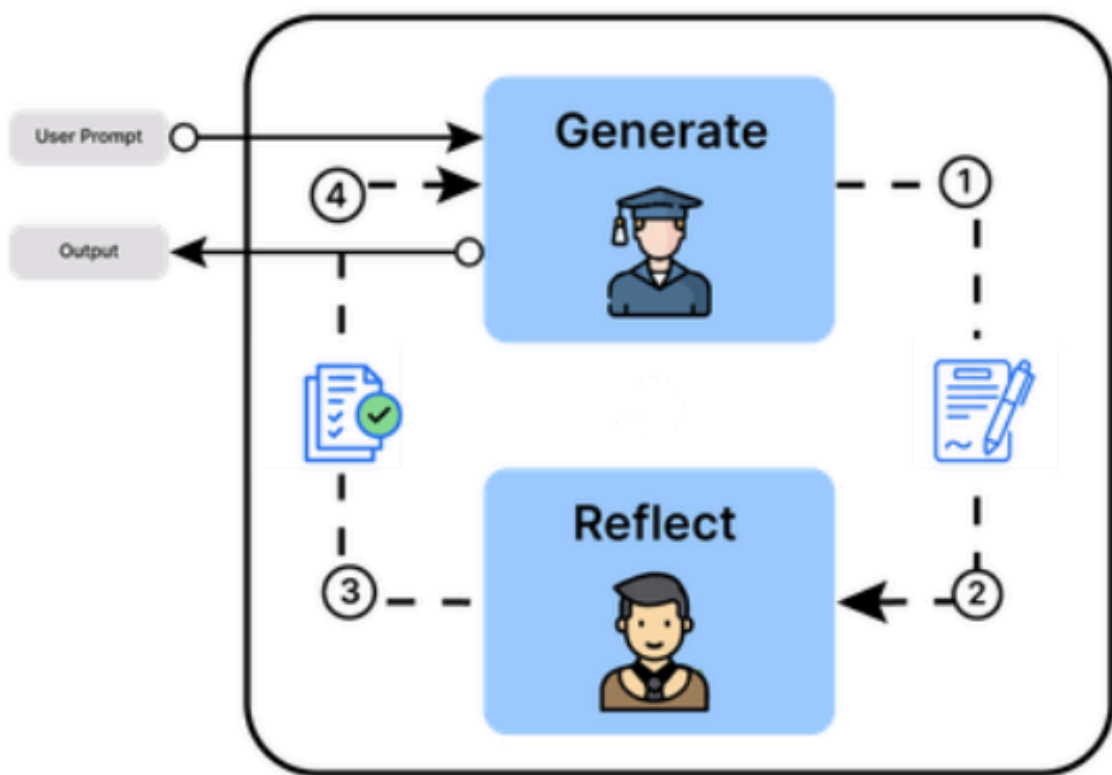
LLMs do well at this, but they can do even better. By using an agent-like workflow, we can prompt the LLM multiple times in steps. Each step builds on the previous one, refining the response. Think of it like **asking the LLM to go over the essay multiple times**, improving it with each pass.

4 Types of Agentic Design Patterns

In Agentic AI and the key design patterns, it's essential to understand how each pattern empowers large language models (LLMs) like GPT to behave more autonomously and effectively. These design patterns push the boundaries of what AI can do by encouraging self-evaluation, tool integration, strategic thinking, and collaboration. Let's explore four vital agentic design patterns that shape how these models operate and perform complex tasks.



Reflection Pattern



The Reflection Pattern focuses on improving AI's ability to evaluate and refine its own outputs. Imagine an LLM reviewing its generated content or code as if it were a human reviewer, identifying errors, gaps, or areas that need improvement and then offering suggestions for how to improve.

This self-critique loop **is not limited to a single iteration**. The AI can repeat the reflection process as many times as necessary to achieve a refined, polished result. This pattern is particularly useful in tasks that require precision, such as:

Content Creation

Problem Solving

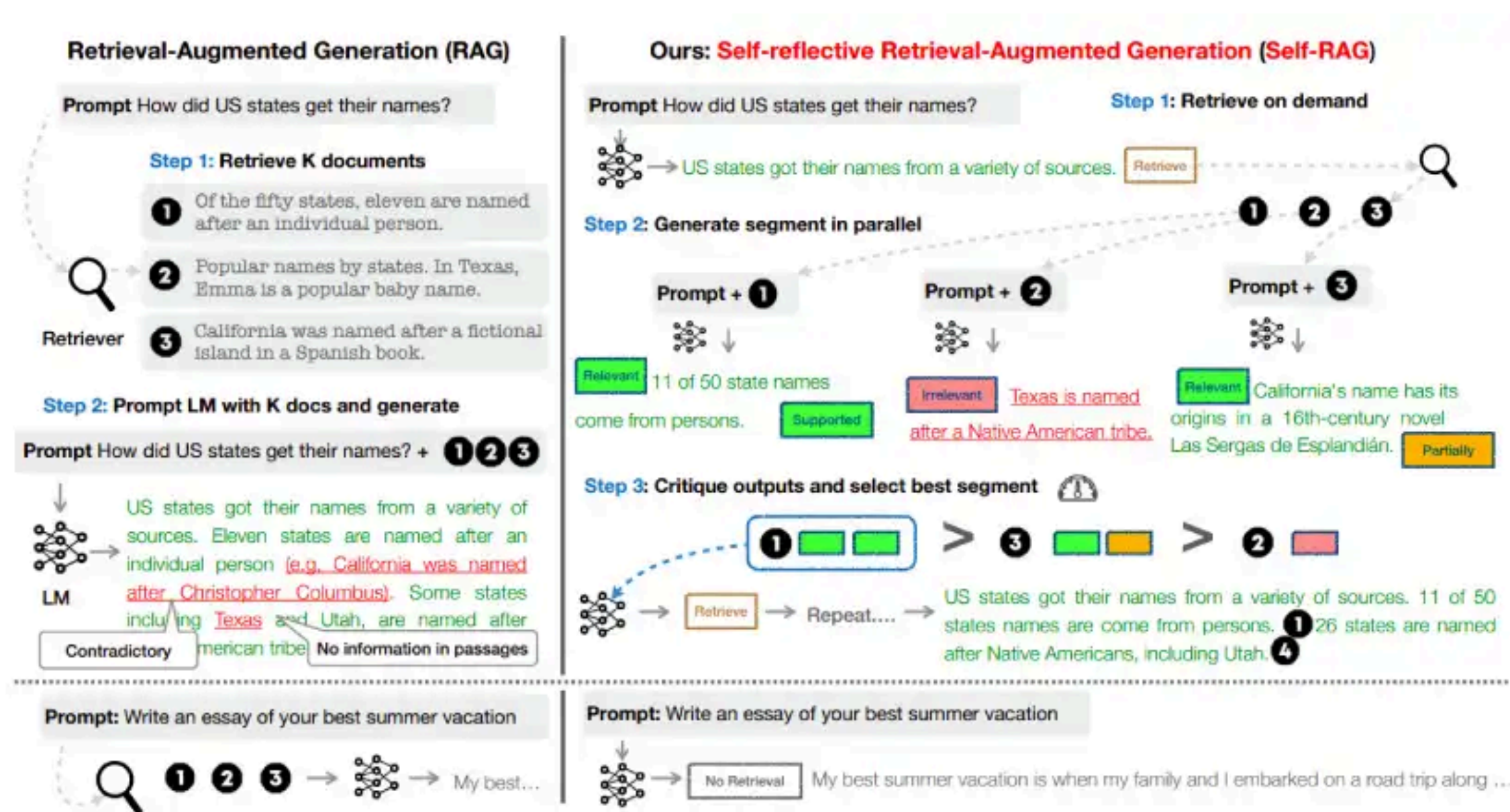
Code Generation

One interesting example is Self-Reflective RAG. [SELF-RAG](#) is a framework designed to improve language models' quality and factual accuracy by integrating retrieval and self-reflection into the text generation process.

SELF-RAG incorporates **self-reflection mechanisms** via “reflection tokens,” which serve to evaluate various aspects of the text generation, such as relevance, support, and overall utility.

During the generation process, the model evaluates whether retrieval is necessary and assesses the quality of the generated content by critiquing itself at different stages.

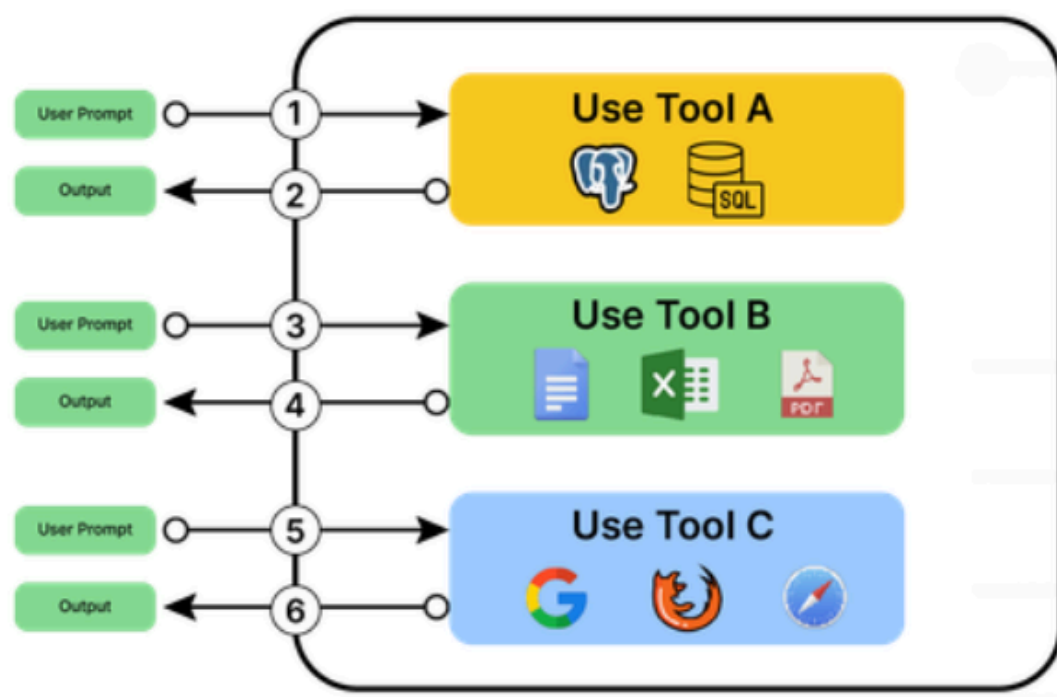
Here’s the diagram for better understanding:



In a nutshell, **Self-RAG** adds an extra layer of **self-reflection** and **refinement**, leading to more reliable and precise answers.

Tool Use Pattern

The Tool Use Pattern significantly **broadens an LLM's capability** by allowing it to interact with external tools and resources to enhance its problem-solving abilities.



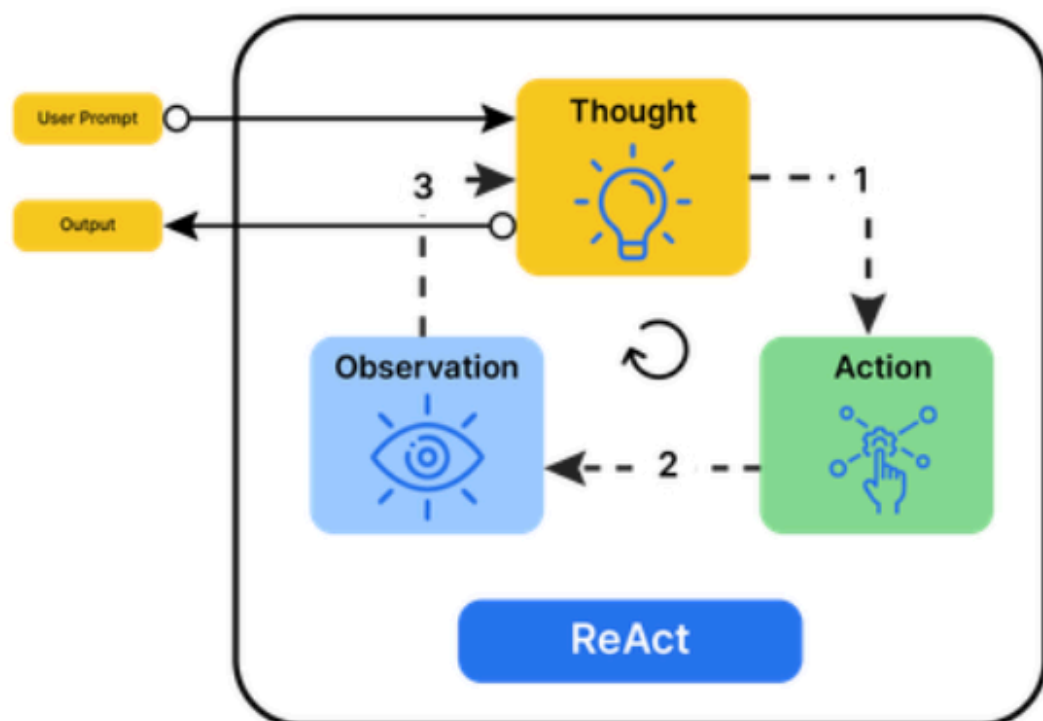
Instead of relying solely on internal computations or knowledge, an AI following this pattern can **access databases, search the web, or even execute complex functions** via programming languages like Python.

For instance, an LLM could be prompted to retrieve data from the web for a specific query, analyze it, and integrate it into its output.

Alternatively, it might be tasked with calculating statistical results, generating images, or manipulating spreadsheets actions that go beyond simple text generation. By incorporating the use of tools, LLMs evolve from static knowledge banks into dynamic agents capable of interacting with external systems to achieve goals.

This pattern is powerful because it allows AI systems to tackle more complex, multifaceted tasks where internal knowledge alone isn't sufficient, expanding their utility into real-world applications.

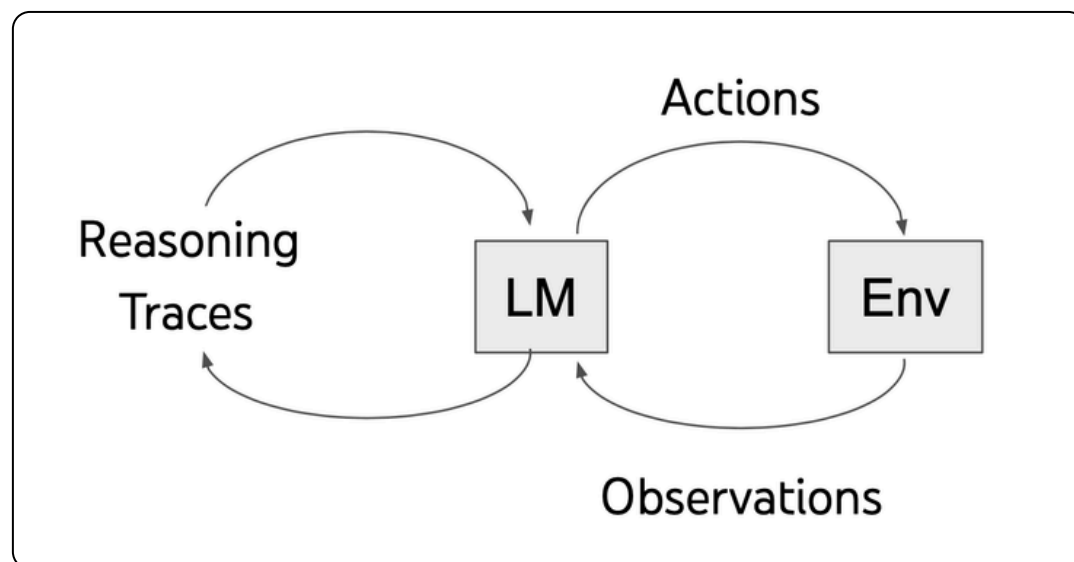
Planning Pattern



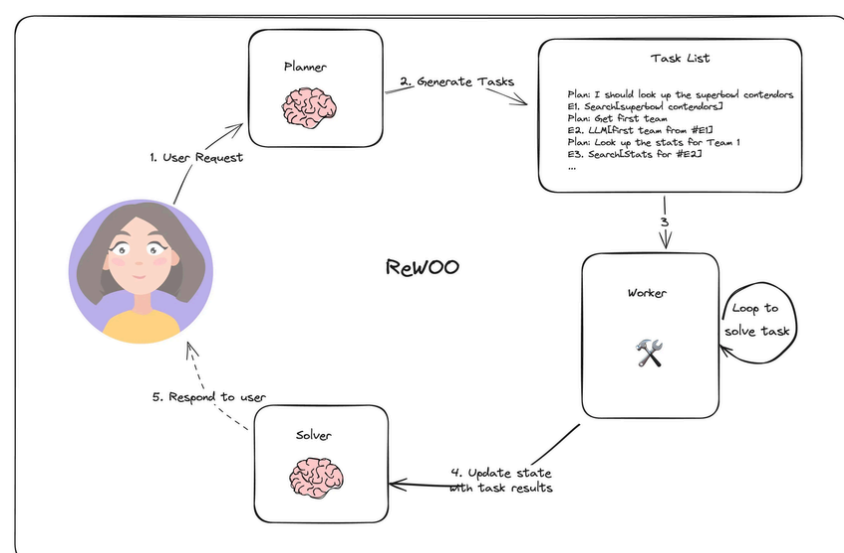
The Planning Pattern enables an LLM to break down **large, complicated tasks** into **smaller, more manageable components**. Planning equips an agent with the ability to react to requests and strategically structure the steps needed to achieve a goal.

For example, when coding, the LLM would first outline the overall structure before implementing individual functions. This avoids confusion or meandering logic and keeps the AI focused on the main objective.

ReAct(Reasoning + Acting)

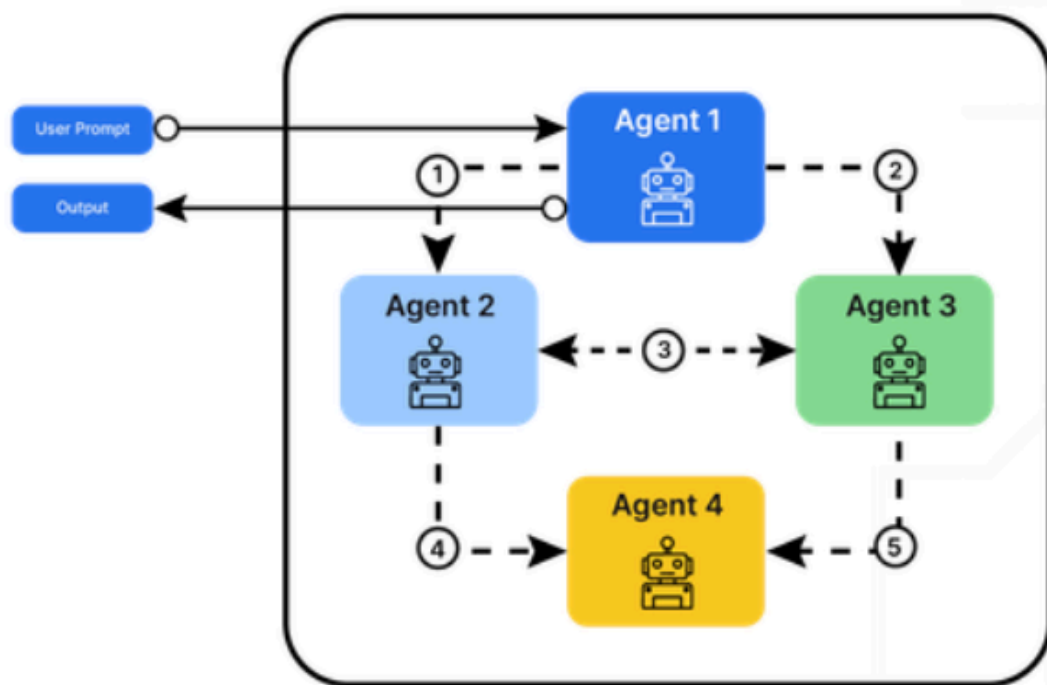


ReWOO (Reasoning With Open Ontology)



further extend this approach by integrating decision-making and contextual reasoning into the planning process. ReAct enables the LLM to dynamically alternate between reasoning (thinking through the problem) and acting (performing specific tasks), allowing for more adaptive and flexible planning. By combining these two steps, the LLM can refine its approach iteratively, addressing unexpected challenges as they arise.

Multi-Agent Pattern



The Multi-Agent Pattern builds upon the **concept of delegation**, akin to project management in human teams. This pattern involves assigning **different agents** (which are instances of an LLM with specific roles or functions) to handle various subtasks.

These agents can work independently on their assignments while also communicating and collaborating to achieve a unified outcome.

There are several types of multi-agent patterns:

Collaborative Agents

Multiple agents work together on different parts of a task, sharing progress and building toward a unified result. Each agent may specialize in a different domain.

Supervised Agents

A central supervisor agent manages other agents, coordinating their activities and verifying results to ensure quality.

Hierarchical Teams

A structured system where higher-level agents oversee lower-level agents, with decision-making cascaded through levels to accomplish complex tasks.

In short, the Multiagent Pattern mirrors how humans collaborate across specialities, ensuring that each agent focuses on its strengths while contributing to a greater, coordinated effort.